

NANYANG TECHNOLOGICAL UNIVERSITY

Data Structures & Algorithms in Python

Analysis of Algorithms – Lab Answers

College of Computing and Data Science

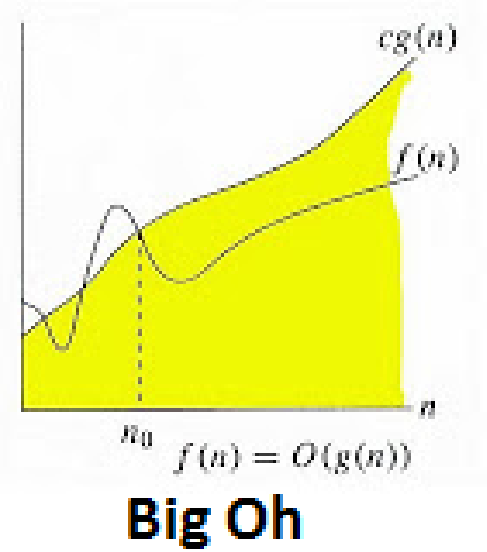
Analysis of Algorithms

- Two considerations:
  - Time complexity
  - Space complexity

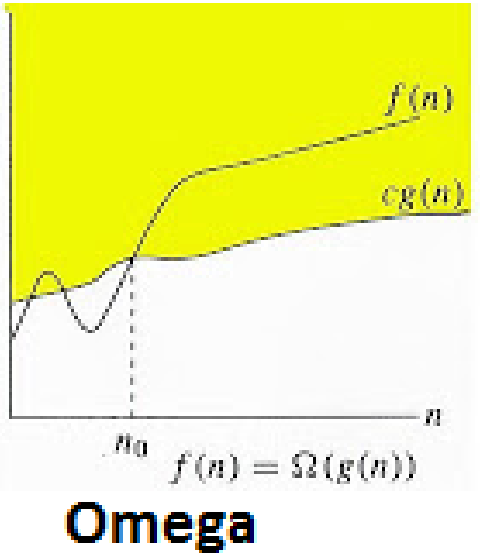
Content Copyright Nanyang Technological University

Analysis of Algorithms

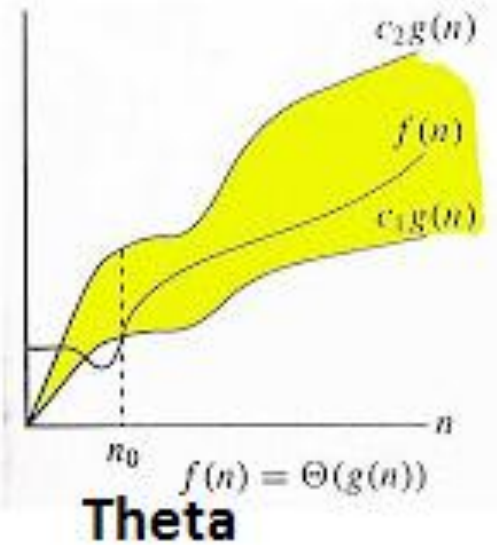
- O()?
  - Upper bound
- Ω()?
  - Lower bound
- Θ()?
  - Tight bound



Big Oh



Omega

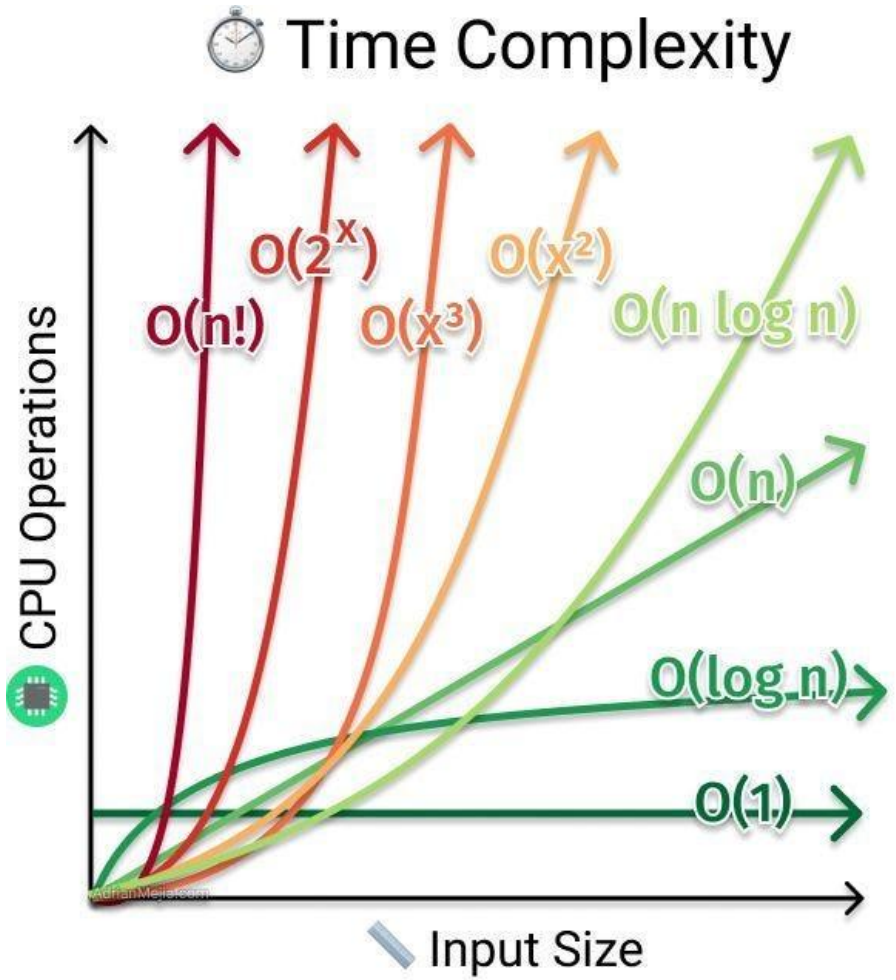


Theta

Content Copyright Nanyang Technological University

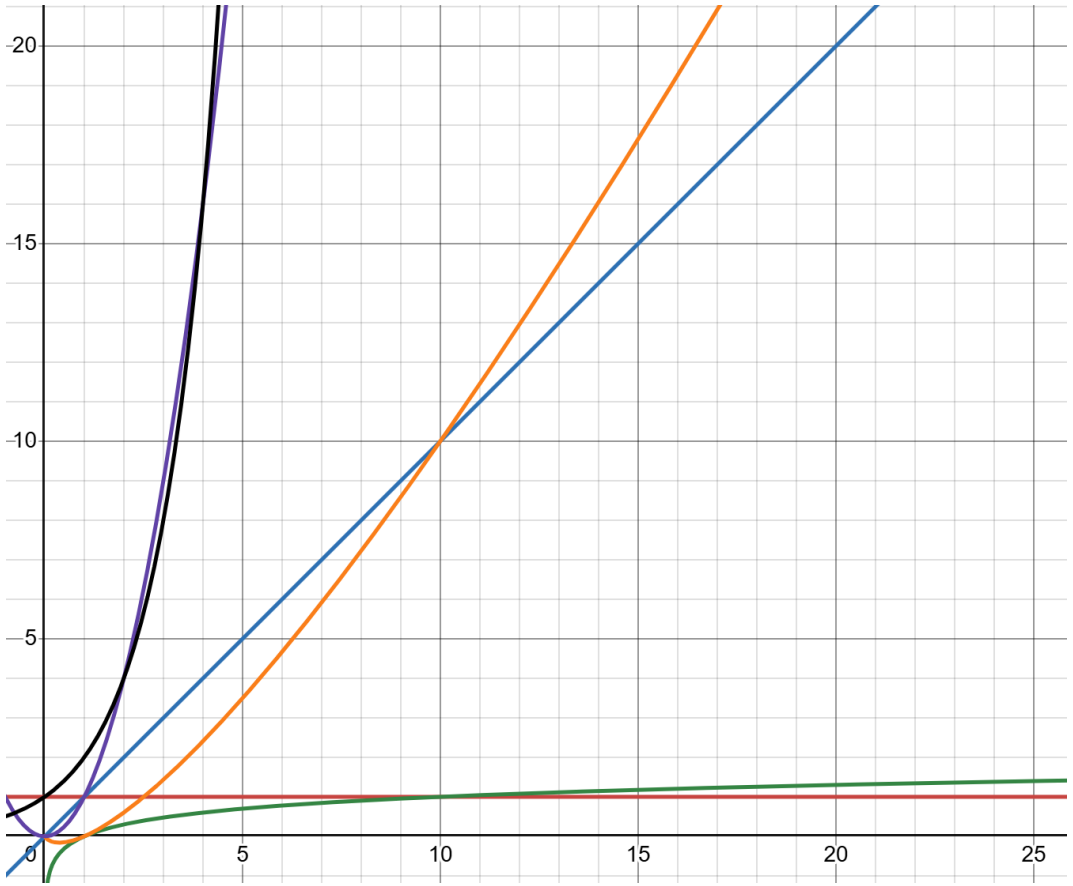
Analysis of Algorithms

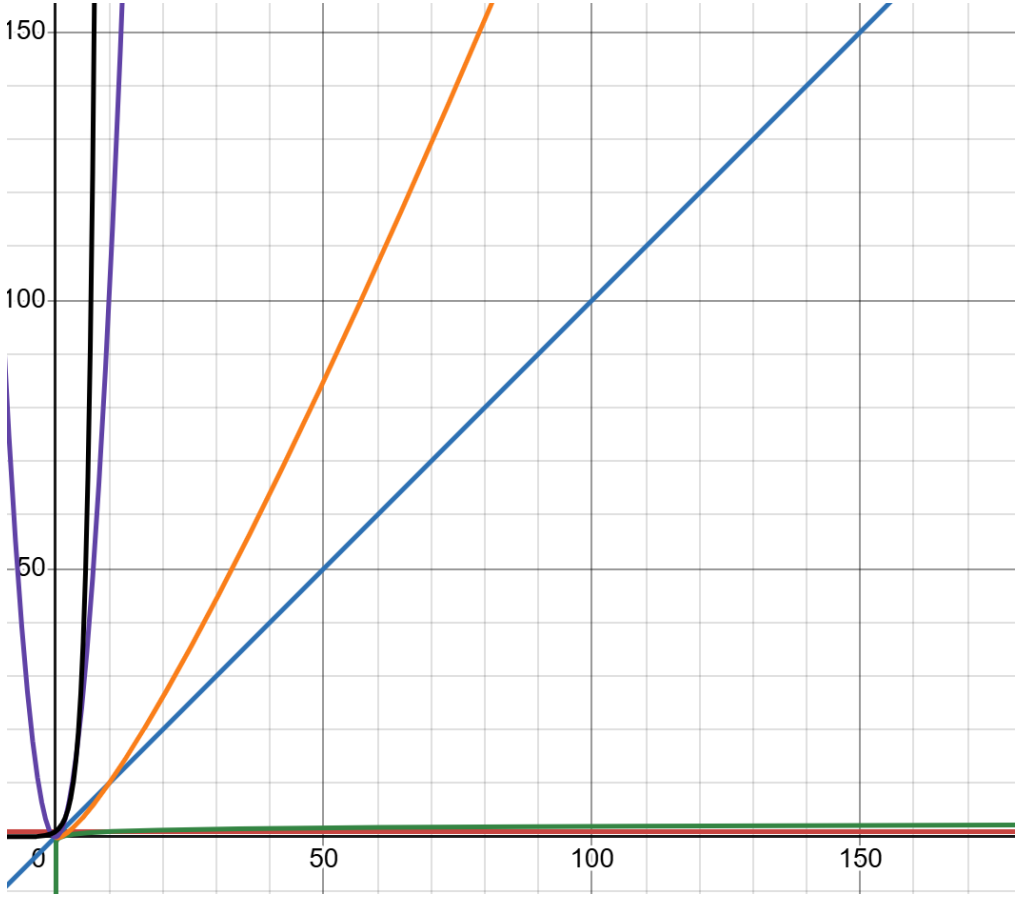
- Exaggerated version



Content Copyright Nanyang Technological University

Analysis of Algorithms





Content Copyright Nanyang Technological University

Analysis of Algorithms

- Ignore coefficients!

	$O(100n)$	$O(n^2)$	$O(100n^2)$	$O(n^3)$
n = 1	10^2	10^0	10^0	10^0
n = 10	10^3	10^2	10^4	10^3
n = 100	10^4	10^4	10^6	10^6
n = 1000	10^5	10^6	10^8	10^9
n = 10000	10^6	10^8	10^{10}	10^{12}

Content Copyright Nanyang Technological University

Analysis of Algorithms

- Ignore smaller exponents!

	$O(n^2)$	$O(n^2+n)$	$O(n^2+100n)$	$O(n^3)$
n = 1	1	2	101	1
n = 10	100	101	1,100	1,000
n = 100	10,000	10,100	20,000	1,000,000
n = 1000	1,000,000	1,001,000	1,100,000	1,000,000,000
n = 10000	100,000,000	100,010,000	101,000,000	10^{12}

Content Copyright Nanyang Technological University

Q1 (No AI)

The Fibonacci numbers are defined by:

$F(0) = 0$, $F(1) = 1$ and $F(n) = F(n - 1) + F(n - 2)$ for $n \geq 2$

Computing them with plain recursion repeats the same subproblems over and over and takes exponential time. Dynamic programming avoids this by solving each subproblem only once. Implement fibonacci(n) using either of the two dynamic programming approaches:

- Memoisation (top-down): recurse as usual, but store each result in a dictionary or list and reuse it instead of recomputing it.
- Tabulation (bottom-up): fill a table from F(0) upwards until you reach F(n), without recursion.

Your solution must run in O(n) time.

Input

n (int) — a non-negative integer

For example, given n = 10:

F(0), F(1), ..., F(10) are 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55

Output

A single integer — the n-th Fibonacci number.

For the example above:

55

Some test cases use large values of n, where plain recursion without memoisation or tabulation would be far too slow.

Content Copyright Nanyang Technological University

Q2 (Normal)

You are in charge of a single lecture theatre and have received requests for several events. Each event has a start time and an end time, and the theatre can only host one event at a time. Your task is to schedule as many events as possible using a greedy algorithm. The greedy choice is to always pick the event that finishes earliest among the events that are still compatible with what you have already scheduled. Two events are compatible if one starts at or after the time the other ends, so an event may start at exactly the moment the previous one finishes.

To make the answer unique, follow these rules:

- Consider the events in order of increasing end time
- If two events end at the same time, consider the one with the smaller index first

Select an event if its start time is at or after the end time of the last event you selected.

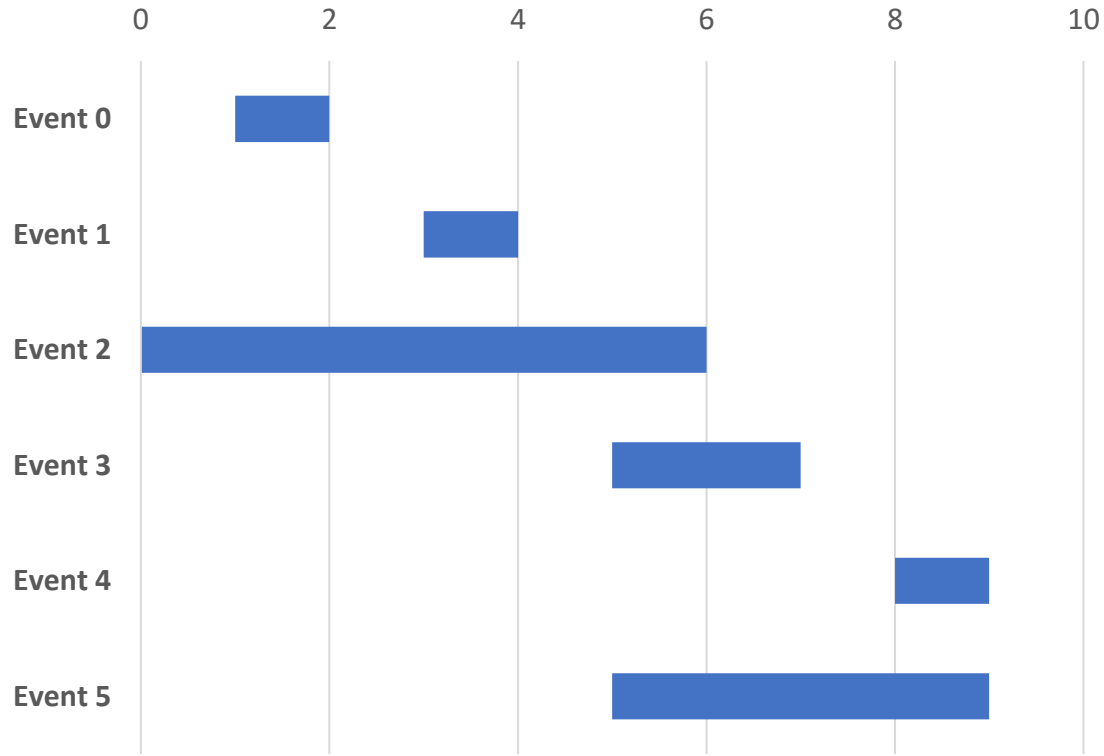
Input:

start_times (list of int) — the start time of each event, all non-negative
end_times (list of int) — the end time of each event, where end_times[i] > start_times[i]

For example, given **start_times** = [1, 3, 0, 5, 8, 5], **end_times** = [2, 4, 6, 7, 9, 9]:

Sorted by end time, the events are considered in the order 0, 1, 2, 3, 4, 5

Event 0 (1–2) is selected, then **event 1** (3–4), since it starts after 2
Event 2 (0–6) starts before 4, so it is skipped; **event 3** (5–7) is selected
Event 4 (8–9) is selected; **event 5** (5–9) starts before 9, so it is skipped



Output:

A list of the indices of the selected events, in the order they were selected.

For the example above:
[0, 1, 3, 4]

Four events can be scheduled: 1–2, 3–4, 5–7 and 8–9. If there are no events, return an empty list.

Content Copyright Nanyang Technological University

Q3 (Driver)

Suppose you are programming an autonomous delivery drone that must deliver packages to multiple locations in a city and return to its starting point (warehouse). The drone must find the shortest possible route that visits each delivery location exactly once before returning to the warehouse. Each location has a distance to every other location. The drone's objective is to minimize the total travel distance. Your task is to implement a brute-force approach to find the shortest distance for the inputs.

Input:

A single distance matrix of size n × n, where distance_matrix[i][j] represents the distance from location i to location j. Location 0 is always the warehouse.

For example, given 4 locations (including the warehouse):

[[0, 10, 15, 20], [10, 0, 35, 25], [15, 35, 0, 30], [20, 25, 30, 0]]

Row 0: distances from the warehouse, 0 to itself, 10 to location 1, 15 to location 2, 20 to location 3. Row 1: distances from location 1, 10 to warehouse, 0 to itself, 35 to location 2, 25 to location 3. and so on...

Output:

A list containing two elements:

The shortest total distance The best route (starting and ending at location 0)

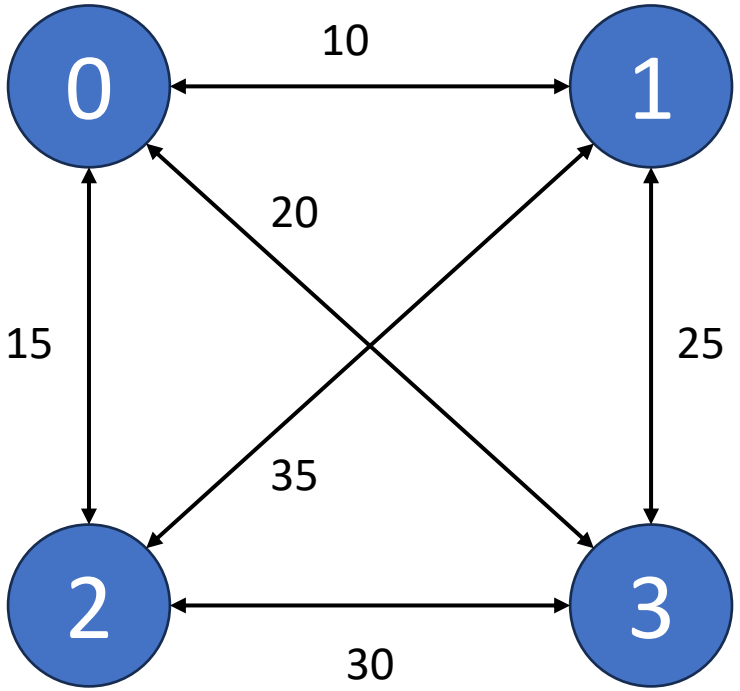
For the example above:
[80, [0, 1, 3, 2, 0]]

The optimal route is 0 → 1 → 3 → 2 → 0 with a total distance of 10 + 25 + 30 + 15 = 80.

Complete the brute_force_tsp function and return a list containing the following values: [**min_distance**, **best_route**]

Content Copyright Nanyang Technological University

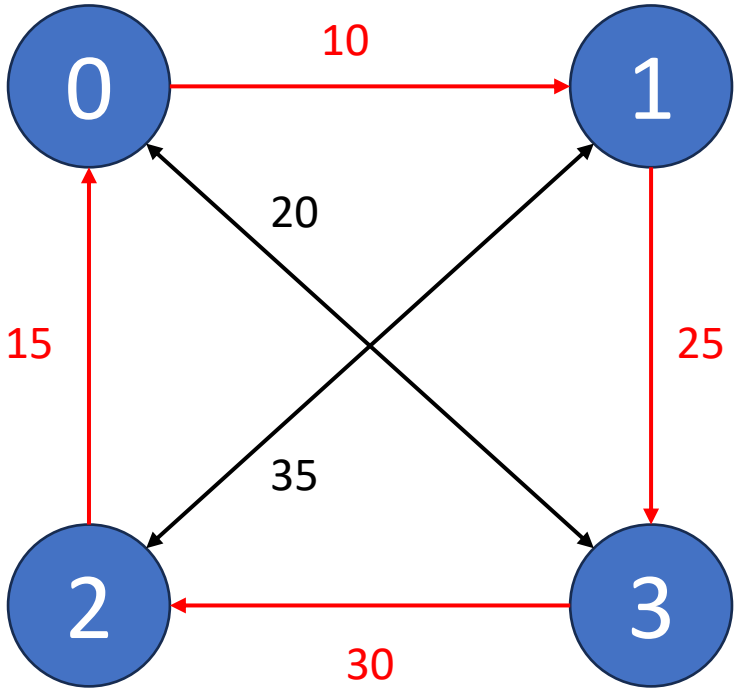
Q3 (Driver)



	0	1	2	3
0	0	10	15	20
1	10	0	35	25
2	15	35	0	30
3	20	25	30	0

Content Copyright Nanyang Technological University

Q3 (Driver)



	0	1	2	3
0	0	10	15	20
1	10	0	35	25
2	15	35	0	30
3	20	25	30	0

Content Copyright Nanyang Technological University

Q4 (Navigator)

Suppose you are a treasure hunter exploring an ancient cave filled with valuable artifacts. You have a backpack with a limited weight capacity W, and you must decide which artifacts to take to maximize your total value. Each artifact has a weight and a value. You cannot take a fraction of an artifact, you must either take it entirely or leave it behind.

Your task is to implement a brute-force approach that tries all possible subsets of artifacts to find the combination that maximizes total value while staying within the weight limit.

Input

Three parameters:

- W (int) - the backpack's weight capacity values
- (list of int) - the value of each artifact weights
- (list of int) - the weight of each artifact

For example, given W = 50, values = [60, 100, 120, 30, 80], weights = [10, 20, 30, 5, 15]:

Artifact	Value	Weight
0	60	10
1	100	20
2	120	30
3	30	5
4	80	15

Output A list containing three elements:

- The maximum total value achievable
- A list of [weight, value] pairs for each selected artifact
- The total weight used

For the example above:

[270, [[10, 60], [20, 100], [5, 30], [15, 80]], 50] The optimal selection is artifacts 0, 1, 3, and 4 with a total value of 60 + 100 + 30 + 80 = 270 and a total weight of 10 + 20 + 5 + 15 = 50.

Content Copyright Nanyang Technological University

Q1 (No AI) – Answer

The Fibonacci numbers are defined by:

$F(0) = 0$, $F(1) = 1$ and $F(n) = F(n - 1) + F(n - 2)$ for $n \geq 2$

Computing them with plain recursion repeats the same subproblems over and over and takes exponential time. Dynamic programming avoids this by solving each subproblem only once. Implement fibonacci(n) using either of the two dynamic programming approaches:

- Memoisation (top-down): recurse as usual, but store each result in a dictionary or list and reuse it instead of recomputing it.
- Tabulation (bottom-up): fill a table from F(0) upwards until you reach F(n), without recursion.

Your solution must run in O(n) time.

Input

n (int) — a non-negative integer

For example, given n = 10:

F(0), F(1), ..., F(10) are 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55

Output

A single integer — the n-th Fibonacci number.

For the example above:

55

Some test cases use large values of n, where plain recursion without memoisation or tabulation would be far too slow.

Content Copyright Nanyang Technological University

Q1 (No AI) – Answer

```python
def fibonacci(self, n: int) -> int:
    if n <= 1:
        return n
    dp = [0] * (n + 1)
    dp[1] = 1
    for i in range(2, n + 1):
        dp[i] = dp[i - 1] + dp[i - 2]
    return dp[n]
```

```python
def fibonacci(self, n: int) -> int:
    def fibonacciMemoisation(n, dp):
        if n <= 1:
            return n
        if dp[n] != -1:
            return dp[n]
        dp[n] = fibonacciMemoisation(n - 1, dp) +
                fibonacciMemoisation(n - 2, dp)
        return dp[n]

    dp = [-1] * (n + 1)
    return fibonacciMemoisation(n, dp)
```

Content Copyright Nanyang Technological University

Q1 (No AI) – Answer

```python
def fibonacci(self, n: int) -> int:
    if n <= 1:
        return n
    dp = [0] * (n + 1)
    dp[1] = 1
    for i in range(2, n + 1):
        dp[i] = dp[i - 1] + dp[i - 2]
    return dp[n]
```

Content Copyright Nanyang Technological University

Q1 (No AI) – Answer

```
def fibonacci(self, n: int) -> int:
    if n <= 1:
        return n
    dp = [0] * (n + 1)
    dp[1] = 1
    for i in range(2, n + 1):
        dp[i] = dp[i - 1] + dp[i - 2]
    return dp[n]
```

Content Copyright Nanyang Technological University

Q1 (No AI) – Answer

```python
def fibonacci(self, n: int) -> int:
    if n <= 1:
        return n
    dp = [0] * (n + 1)
    dp[1] = 1
    for i in range(2, n + 1):
        dp[i] = dp[i - 1] + dp[i - 2]
    return dp[n]
```

dp[0]	0
dp[1]	0
dp[2]	0
dp[3]	0
dp[4]	0
dp[5]	0

Content Copyright Nanyang Technological University

Q1 (No AI) – Answer

```
def fibonacci(self, n: int) -> int:
    if n <= 1:
        return n
    dp = [0] * (n + 1)
    dp[1] = 1
    for i in range(2, n + 1):
        dp[i] = dp[i - 1] + dp[i - 2]
    return dp[n]
```

dp[0]	0
dp[1]	1
dp[2]	0
dp[3]	0
dp[4]	0
dp[5]	0

Content Copyright Nanyang Technological University

Q1 (No AI) – Answer

```
def fibonacci(self, n: int) -> int:
    if n <= 1:
        return n
    dp = [0] * (n + 1)
    dp[1] = 1
    for i in range(2, n + 1):
        dp[i] = dp[i - 1] + dp[i - 2]
    return dp[n]
```

i = 2

dp[0]	0
dp[1]	1
dp[2]	1
dp[3]	0
dp[4]	0
dp[5]	0

Content Copyright Nanyang Technological University

Q1 (No AI) – Answer

```python
def fibonacci(self, n: int) -> int:
    if n <= 1:
        return n
    dp = [0] * (n + 1)
    dp[1] = 1
    for i in range(2, n + 1):
        dp[i] = dp[i - 1] + dp[i - 2]
    return dp[n]
```

i = 3

dp[0]	0
dp[1]	1
dp[2]	1
dp[3]	2
dp[4]	0
dp[5]	0

Content Copyright Nanyang Technological University

Q1 (No AI) – Answer

```python
def fibonacci(self, n: int) -> int:
    if n <= 1:
        return n
    dp = [0] * (n + 1)
    dp[1] = 1
    for i in range(2, n + 1):
        dp[i] = dp[i - 1] + dp[i - 2]
    return dp[n]
```

i = 4

dp[0]	0
dp[1]	1
dp[2]	1
dp[3]	2
dp[4]	3
dp[5]	0

Content Copyright Nanyang Technological University

Q1 (No AI) – Answer

```python
def fibonacci(self, n: int) -> int:
    if n <= 1:
        return n
    dp = [0] * (n + 1)
    dp[1] = 1
    for i in range(2, n + 1):
        dp[i] = dp[i - 1] + dp[i - 2]
    return dp[n]
```

i = 5

dp[0]	0
dp[1]	1
dp[2]	1
dp[3]	2
dp[4]	3
dp[5]	5

Content Copyright Nanyang Technological University

Q1 (No AI) – Answer

```python
def fibonacci(self, n: int) -> int:
    if n <= 1:
        return n
    dp = [0] * (n + 1)
    dp[1] = 1
    for i in range(2, n + 1):
        dp[i] = dp[i - 1] + dp[i - 2]
    return dp[n]
```

Content Copyright Nanyang Technological University

Q1 (No AI) – Answer

```python
def fibonacci(self, n: int) -> int:
    if n <= 1:
        return n
    dp = [0] * (n + 1)
    dp[1] = 1
    for i in range(2, n + 1):
        dp[i] = dp[i - 1] + dp[i - 2]
    return dp[n]
```

O(4) → O(1)

O(1)

Content Copyright Nanyang Technological University

Q1 (No AI) – Answer

```
def fibonacci(self, n: int) -> int:
    if n <= 1:
        return n
    dp = [0] * (n + 1)
    dp[1] = 1
    for i in range(2, n + 1):
        dp[i] = dp[i - 1] + dp[i - 2]
    return dp[n]
```

O(1) + O(n) → O(n)

O(1) + O(n)

Content Copyright Nanyang Technological University

Q1 (No AI) – Answer

```python
def fibonacci(self, n: int) -> int:
    if n <= 1:
        return n
    dp = [0] * (n + 1)
    dp[1] = 1
    for i in range(2, n + 1):
        dp[i] = dp[i - 1] + dp[i - 2]
    return dp[n]
```

O(1) + O(n) → O(n)

Content Copyright Nanyang Technological University

Q1 (No AI) – Answer

```
def fibonacci(self, n: int) -> int:
    def fibonacciMemoisation(n, dp):
        if n <= 1:
            return n
        if dp[n] != -1:
            return dp[n]
        dp[n] = fibonacciMemoisation(n - 1, dp) +
                fibonacciMemoisation(n - 2, dp)
        return dp[n]

    dp = [-1] * (n + 1)
    return fibonacciMemoisation(n, dp)
```

Content Copyright Nanyang Technological University

Q1 (No AI) – Answer

```
def fibonacci(self, n: int) -> int:
    def fibonacciMemoisation(n, dp):
        if n <= 1:
            return n
        if dp[n] != -1:
            return dp[n]
        dp[n] = fibonacciMemoisation(n - 1, dp) +
                fibonacciMemoisation(n - 2, dp)
        return dp[n]

    dp = [-1] * (n + 1)
    return fibonacciMemoisation(n, dp)
```

Content Copyright Nanyang Technological University

Q1 (No AI) – Answer

```
def fibonacci(self, n: int) -> int:
    def fibonacciMemoisation(n, dp):
        if n <= 1:
            return n
        if dp[n] != -1:
            return dp[n]
        dp[n] = fibonacciMemoisation(n - 1, dp) +
                fibonacciMemoisation(n - 2, dp)
        return dp[n]

    dp = [-1] * (n + 1)
    return fibonacciMemoisation(n, dp)
```

Content Copyright Nanyang Technological University

Q1 (No AI) – Answer

```
def fibonacci(self, n: int) -> int:
    def fibonacciMemoisation(n, dp):
        if n <= 1:
            return n
        if dp[n] != -1:
            return dp[n]
        dp[n] = fibonacciMemoisation(n - 1, dp) +
                fibonacciMemoisation(n - 2, dp)
        return dp[n]

    dp = [-1] * (n + 1)
    return fibonacciMemoisation(n, dp)
```

dp[0]	-1
dp[1]	-1
dp[2]	-1
dp[3]	-1
dp[4]	-1
dp[5]	-1

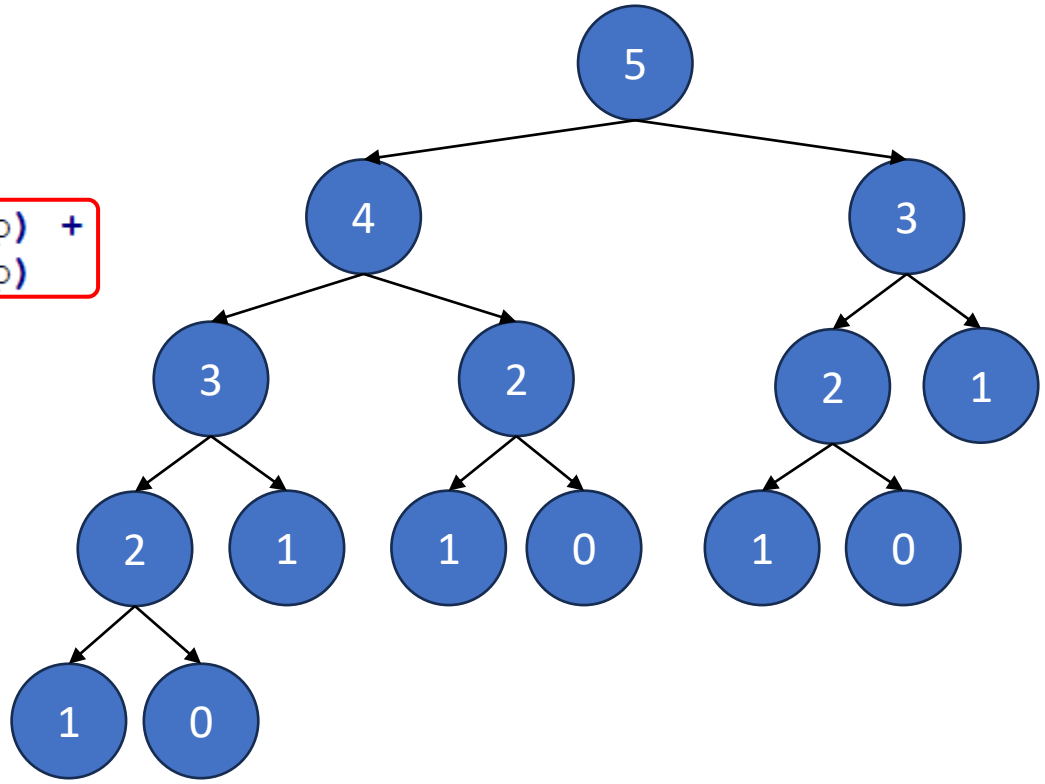
Content Copyright Nanyang Technological University

Q1 (No AI) – Answer

```
def fibonacci(self, n: int) -> int:
    def fibonacciMemoisation(n, dp):
        if n <= 1:
            return n
        if dp[n] != -1:
            return dp[n]
        dp[n] = fibonacciMemoisation(n - 1, dp) +
                fibonacciMemoisation(n - 2, dp)
        return dp[n]

    dp = [-1] * (n + 1)
    return fibonacciMemoisation(n, dp)
```

dp[0]	-1
dp[1]	-1
dp[2]	-1
dp[3]	-1
dp[4]	-1
dp[5]	-1



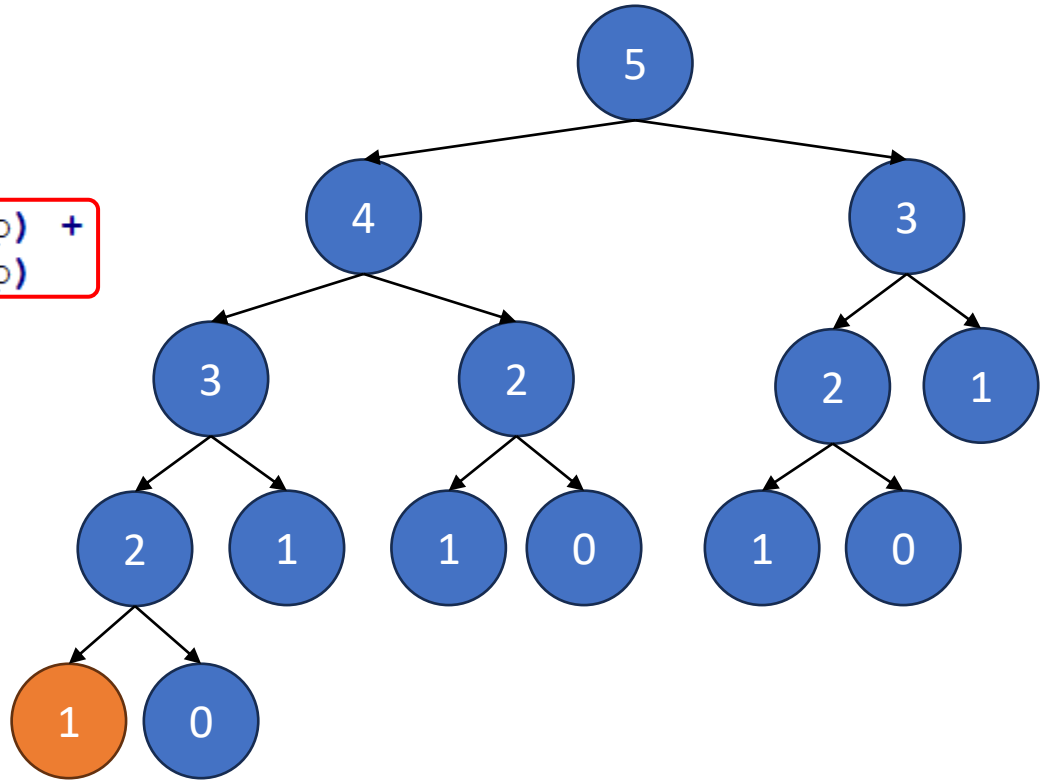
Content Copyright Nanyang Technological University

Q1 (No AI) – Answer

```
def fibonacci(self, n: int) -> int:
    def fibonacciMemoisation(n, dp):
        if n <= 1:
            return n
        if dp[n] != -1:
            return dp[n]
        dp[n] = fibonacciMemoisation(n - 1, dp) +
                fibonacciMemoisation(n - 2, dp)
        return dp[n]

    dp = [-1] * (n + 1)
    return fibonacciMemoisation(n, dp)
```

dp[0]	-1
dp[1]	1
dp[2]	-1
dp[3]	-1
dp[4]	-1
dp[5]	-1



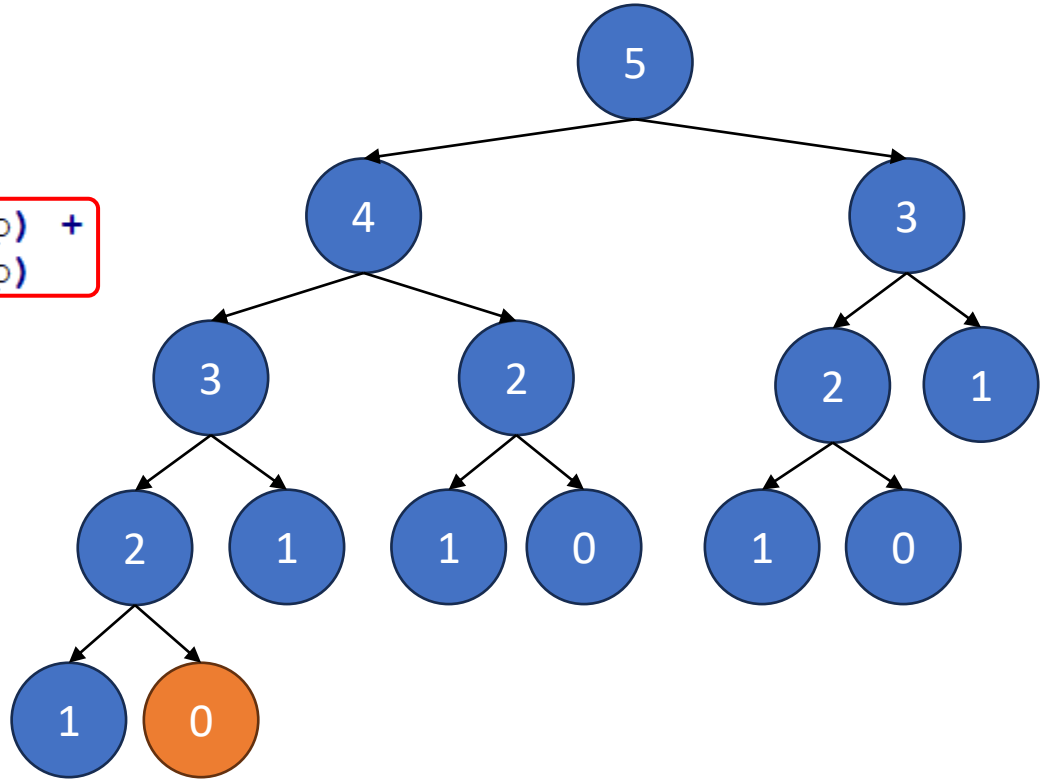
Content Copyright Nanyang Technological University

Q1 (No AI) – Answer

```
def fibonacci(self, n: int) -> int:
    def fibonacciMemoisation(n, dp):
        if n <= 1:
            return n
        if dp[n] != -1:
            return dp[n]
        dp[n] = fibonacciMemoisation(n - 1, dp) +
                fibonacciMemoisation(n - 2, dp)
        return dp[n]

    dp = [-1] * (n + 1)
    return fibonacciMemoisation(n, dp)
```

dp[0]	0
dp[1]	1
dp[2]	-1
dp[3]	-1
dp[4]	-1
dp[5]	-1



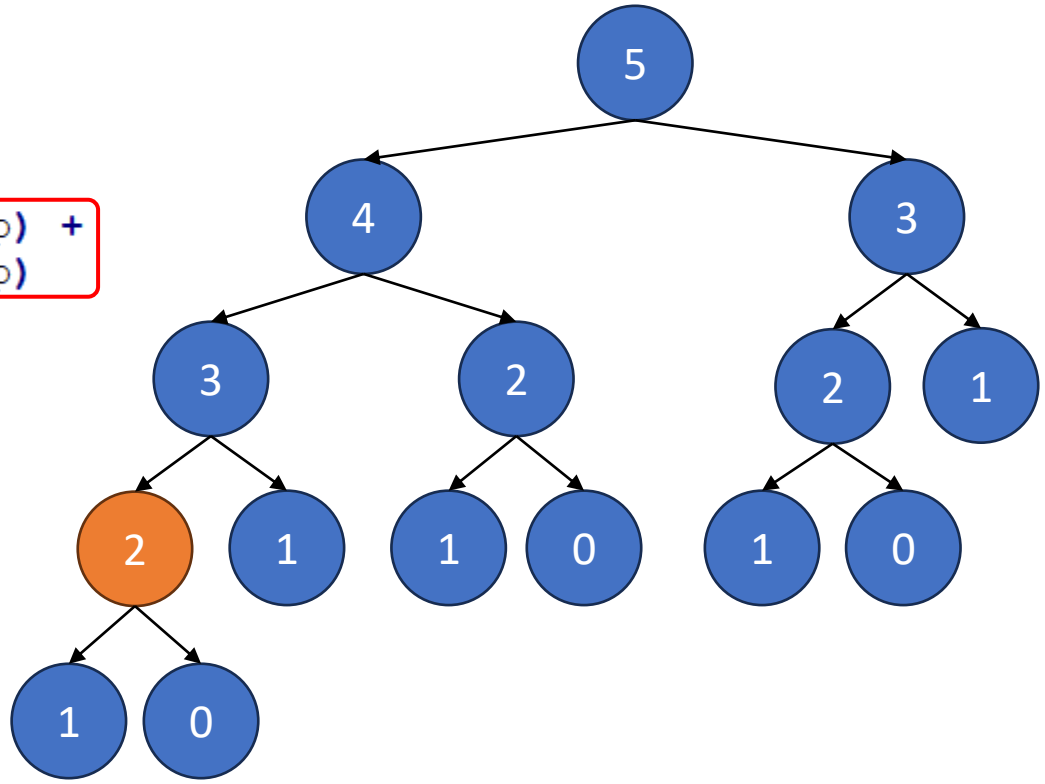
Content Copyright Nanyang Technological University

Q1 (No AI) – Answer

```
def fibonacci(self, n: int) -> int:
    def fibonacciMemoisation(n, dp):
        if n <= 1:
            return n
        if dp[n] != -1:
            return dp[n]
        dp[n] = fibonacciMemoisation(n - 1, dp) +
                fibonacciMemoisation(n - 2, dp)
        return dp[n]

    dp = [-1] * (n + 1)
    return fibonacciMemoisation(n, dp)
```

dp[0]	0
dp[1]	1
dp[2]	1
dp[3]	-1
dp[4]	-1
dp[5]	-1



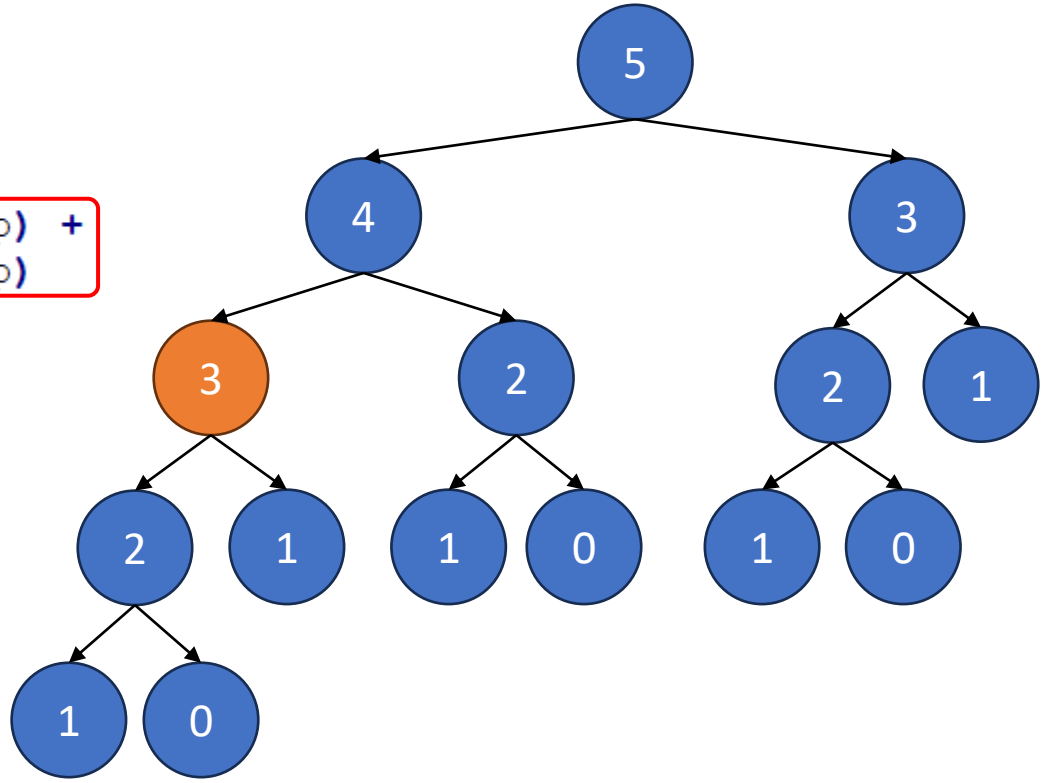
Content Copyright Nanyang Technological University

Q1 (No AI) – Answer

```
def fibonacci(self, n: int) -> int:
    def fibonacciMemoisation(n, dp):
        if n <= 1:
            return n
        if dp[n] != -1:
            return dp[n]
        dp[n] = fibonacciMemoisation(n - 1, dp) +
                fibonacciMemoisation(n - 2, dp)
        return dp[n]

    dp = [-1] * (n + 1)
    return fibonacciMemoisation(n, dp)
```

dp[0]	0
dp[1]	1
dp[2]	1
dp[3]	2
dp[4]	-1
dp[5]	-1



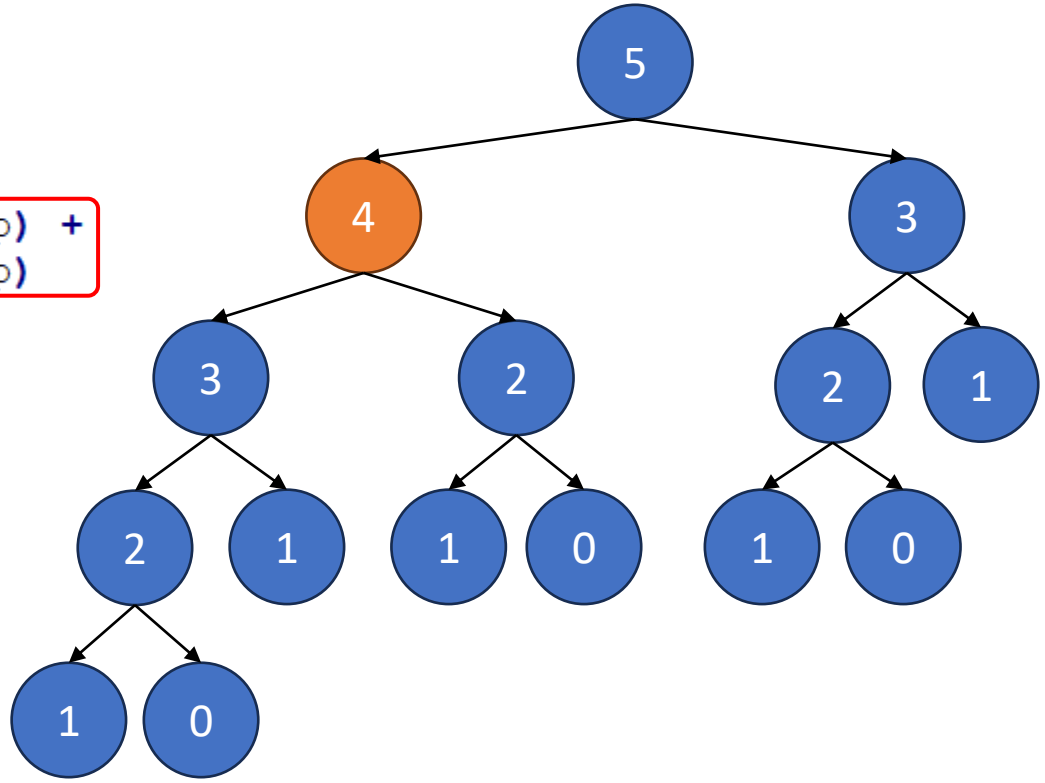
Content Copyright Nanyang Technological University

Q1 (No AI) – Answer

```
def fibonacci(self, n: int) -> int:
    def fibonacciMemoisation(n, dp):
        if n <= 1:
            return n
        if dp[n] != -1:
            return dp[n]
        dp[n] = fibonacciMemoisation(n - 1, dp) +
                fibonacciMemoisation(n - 2, dp)
        return dp[n]

    dp = [-1] * (n + 1)
    return fibonacciMemoisation(n, dp)
```

dp[0]	0
dp[1]	1
dp[2]	1
dp[3]	2
dp[4]	3
dp[5]	-1



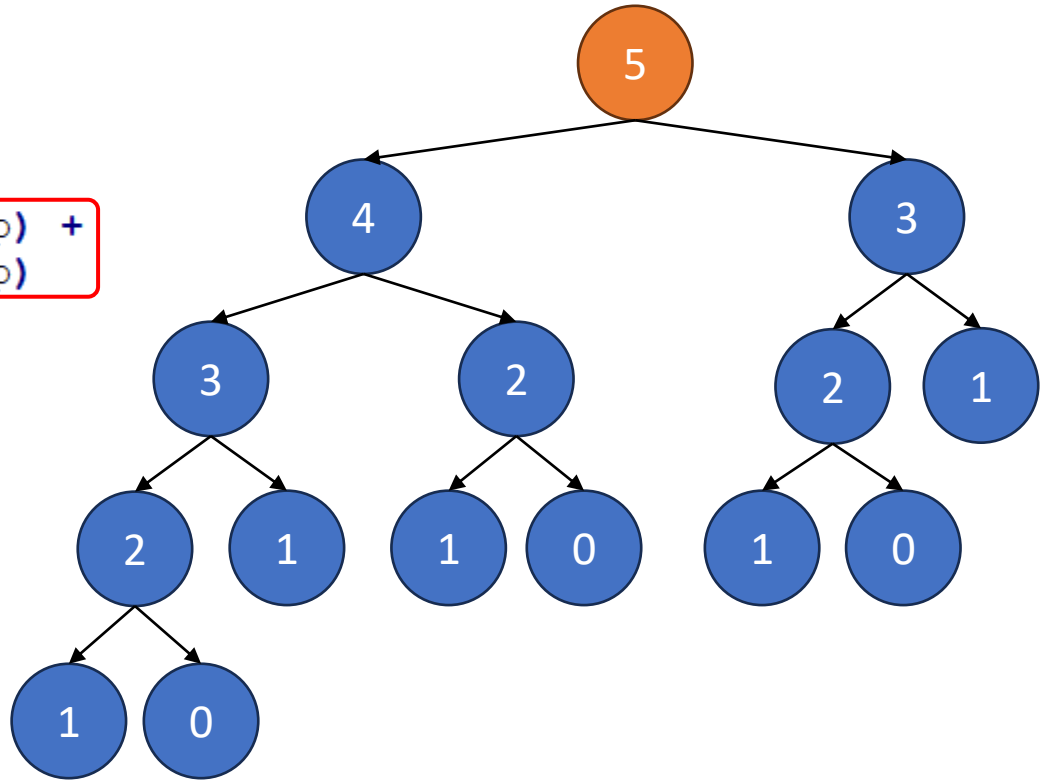
Content Copyright Nanyang Technological University

Q1 (No AI) – Answer

```
def fibonacci(self, n: int) -> int:
    def fibonacciMemoisation(n, dp):
        if n <= 1:
            return n
        if dp[n] != -1:
            return dp[n]
        dp[n] = fibonacciMemoisation(n - 1, dp) +
                fibonacciMemoisation(n - 2, dp)
        return dp[n]

    dp = [-1] * (n + 1)
    return fibonacciMemoisation(n, dp)
```

dp[0]	0
dp[1]	1
dp[2]	1
dp[3]	2
dp[4]	3
dp[5]	5



Content Copyright Nanyang Technological University

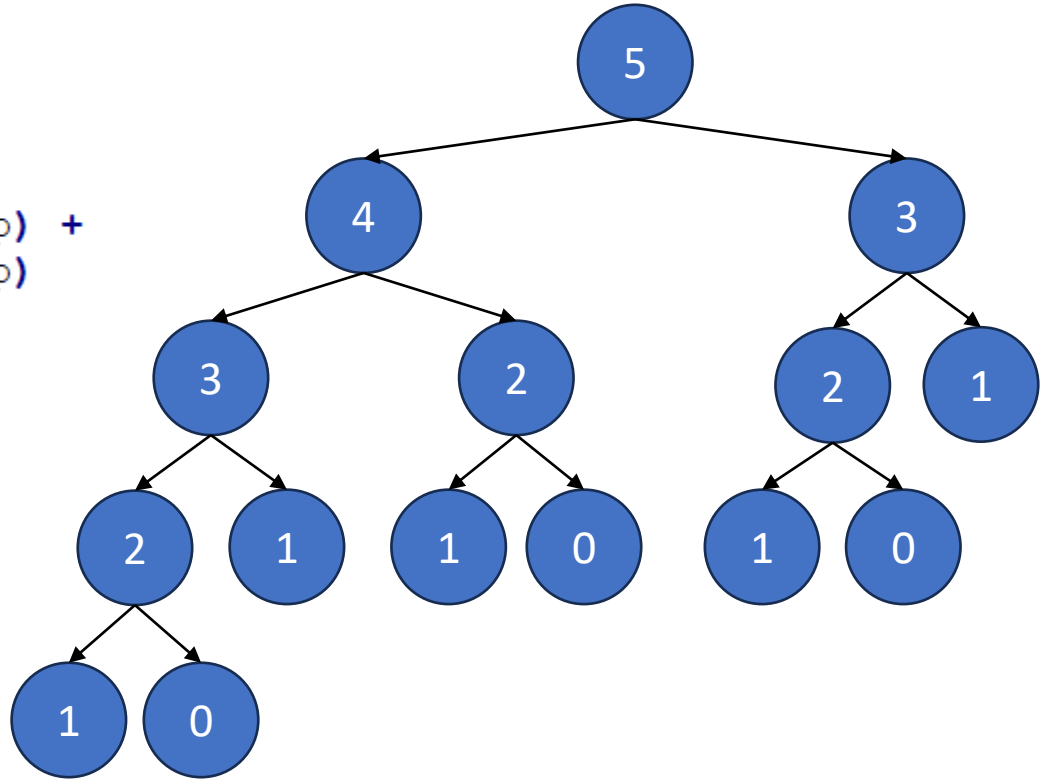
Q1 (No AI) – Answer

```
def fibonacci(self, n: int) -> int:
    def fibonacciMemoisation(n, dp):
        if n <= 1:
            return n
        if dp[n] != -1:
            return dp[n]
        dp[n] = fibonacciMemoisation(n - 1, dp) +
                fibonacciMemoisation(n - 2, dp)
        return dp[n]

    dp = [-1] * (n + 1)
    return fibonacciMemoisation(n, dp)
```

n	calls
0	0
1	0
2	2
3	4
4	8
5	14

$O(1.618^n) \rightarrow O(2^n)$

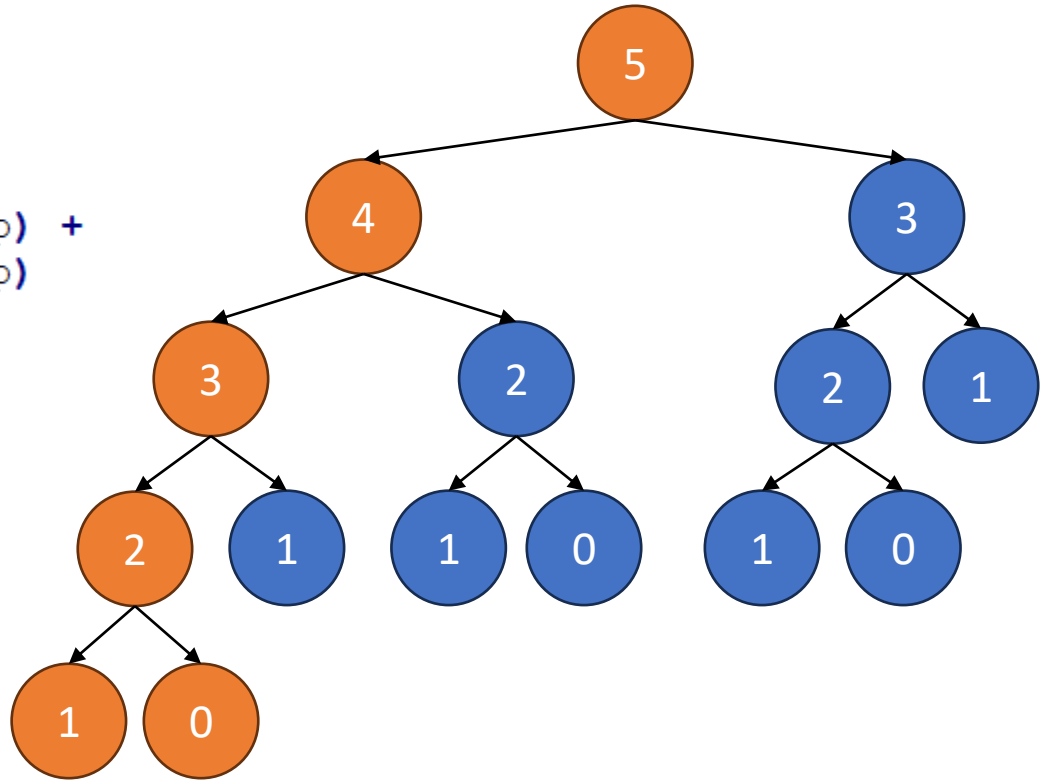


Content Copyright Nanyang Technological University

Q1 (No AI) – Answer

```
def fibonacci(self, n: int) -> int:
    def fibonacciMemoisation(n, dp):
        if n <= 1:
            return n
        if dp[n] != -1:
            return dp[n]
        dp[n] = fibonacciMemoisation(n - 1, dp) +
                fibonacciMemoisation(n - 2, dp)
        return dp[n]

    dp = [-1] * (n + 1)
    return fibonacciMemoisation(n, dp)
```



O(n + 1) → O(n)

Content Copyright Nanyang Technological University

Q2 (Normal) – Answer

You are in charge of a single lecture theatre and have received requests for several events. Each event has a start time and an end time, and the theatre can only host one event at a time. Your task is to schedule as many events as possible using a greedy algorithm. The greedy choice is to always pick the event that finishes earliest among the events that are still compatible with what you have already scheduled. Two events are compatible if one starts at or after the time the other ends, so an event may start at exactly the moment the previous one finishes.

To make the answer unique, follow these rules:

- Consider the events in order of increasing end time
- If two events end at the same time, consider the one with the smaller index first

Select an event if its start time is at or after the end time of the last event you selected.

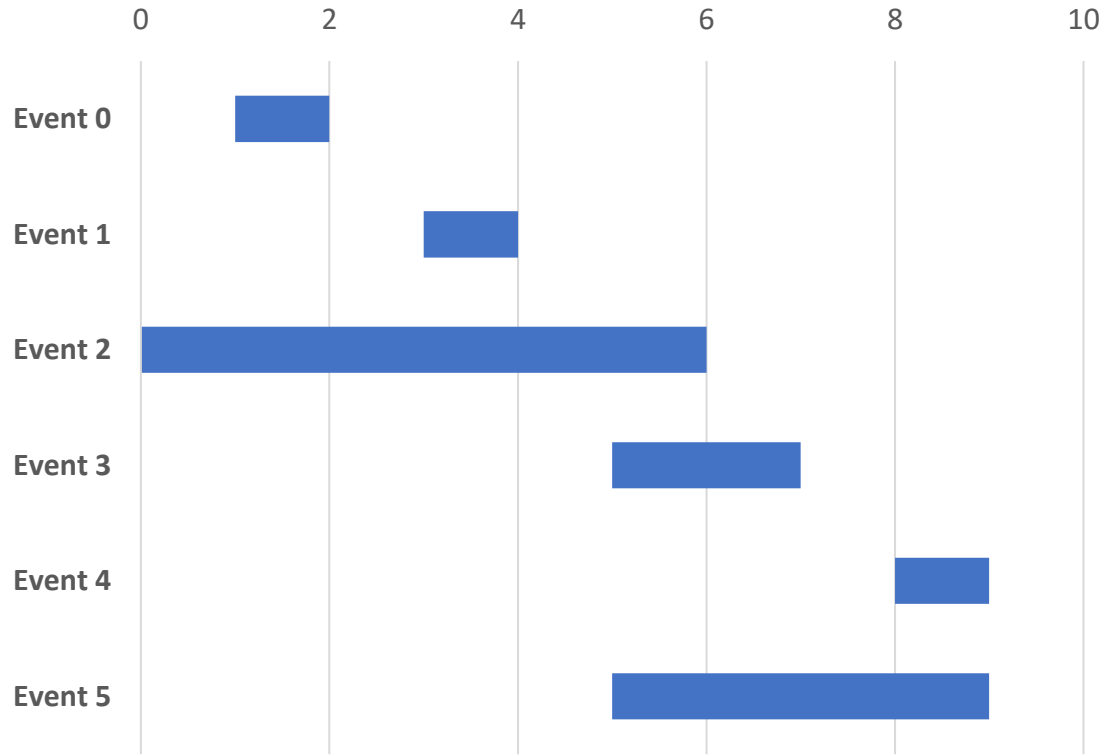
Input:

start_times (list of int) — the start time of each event, all non-negative
end_times (list of int) — the end time of each event, where end_times[i] > start_times[i]

For example, given **start_times** = [1, 3, 0, 5, 8, 5], **end_times** = [2, 4, 6, 7, 9, 9]:

Sorted by end time, the events are considered in the order 0, 1, 2, 3, 4, 5

Event 0 (1–2) is selected, then **event 1** (3–4), since it starts after 2
Event 2 (0–6) starts before 4, so it is skipped; **event 3** (5–7) is selected
Event 4 (8–9) is selected; **event 5** (5–9) starts before 9, so it is skipped



Output:

A list of the indices of the selected events, in the order they were selected.

For the example above:
[0, 1, 3, 4]

Four events can be scheduled: 1–2, 3–4, 5–7 and 8–9. If there are no events, return an empty list.

Content Copyright Nanyang Technological University

Q2 (Normal) – Answer

```
def activity_selection(self, start_times: List[int],
                       end_times: List[int]) -> List[int]:
    activities = list(range(len(start_times)))
    activities.sort(key=lambda i: end_times[i])

    selected = []
    last_end_time = -1

    for i in activities:
        if start_times[i] >= last_end_time:
            selected.append(i)
            last_end_time = end_times[i]

    return selected
```

Content Copyright Nanyang Technological University

Q2 (Normal) – Answer

```
def activity_selection(self, start_times: List[int],
                       end_times: List[int]) -> List[int]:
    activities = list(range(len(start_times)))
    activities.sort(key=lambda i: end_times[i])

    selected = []
    last_end_time = -1

    for i in activities:
        if start_times[i] >= last_end_time:
            selected.append(i)
            last_end_time = end_times[i]

    return selected
```

Activity	Start	End
0	0	6
1	5	9
2	8	9
3	1	2
4	3	4
5	5	7

Content Copyright Nanyang Technological University

Q2 (Normal) – Answer

```
def activity_selection(self, start_times: List[int],
                       end_times: List[int]) -> List[int]:
    activities = list(range(len(start_times)))
    activities.sort(key=lambda i: end_times[i])

    selected = []
    last_end_time = -1

    for i in activities:
        if start_times[i] >= last_end_time:
            selected.append(i)
            last_end_time = end_times[i]

    return selected
```

activities = [0, 1, 2, 3, 4, 5]

Activity	Start	End
0	0	6
1	5	9
2	8	9
3	1	2
4	3	4
5	5	7

Content Copyright Nanyang Technological University

Q2 (Normal) – Answer

```
def activity_selection(self, start_times: List[int],
                       end_times: List[int]) -> List[int]:
    activities = list(range(len(start_times)))
    activities.sort(key=lambda i: end_times[i])

    selected = []
    last_end_time = -1

    for i in activities:
        if start_times[i] >= last_end_time:
            selected.append(i)
            last_end_time = end_times[i]

    return selected
```

Activity	Start	End
0	0	6
1	5	9
2	8	9
3	1	2
4	3	4
5	5	7

activities = [3, 4, 0, 5, 1, 2]

Content Copyright Nanyang Technological University

Q2 (Normal) – Answer

```
def activity_selection(self, start_times: List[int],
                       end_times: List[int]) -> List[int]:
    activities = list(range(len(start_times)))
    activities.sort(key=lambda i: end_times[i])

    selected = []
    last_end_time = -1

    for i in activities:
        if start_times[i] >= last_end_time:
            selected.append(i)
            last_end_time = end_times[i]

    return selected
```

Activity	Start	End
0	0	6
1	5	9
2	8	9
3	1	2
4	3	4
5	5	7

activities = [3, 4, 0, 5, 1, 2]

selected = []

last_end_time = -1

Content Copyright Nanyang Technological University

Q2 (Normal) – Answer

```
def activity_selection(self, start_times: List[int],
                       end_times: List[int]) -> List[int]:
    activities = list(range(len(start_times)))
    activities.sort(key=lambda i: end_times[i])

    selected = []
    last_end_time = -1

    for i in activities:
        if start_times[i] >= last_end_time:
            selected.append(i)
            last_end_time = end_times[i]

    return selected
```

Activity	Start	End
0	0	6
1	5	9
2	8	9
3	1	2
4	3	4
5	5	7

activities = [3, 4, 0, 5, 1, 2]

selected = [3]

last_end_time = 2

Content Copyright Nanyang Technological University

Q2 (Normal) – Answer

```
def activity_selection(self, start_times: List[int],
                       end_times: List[int]) -> List[int]:
    activities = list(range(len(start_times)))
    activities.sort(key=lambda i: end_times[i])

    selected = []
    last_end_time = -1

    for i in activities:
        if start_times[i] >= last_end_time:
            selected.append(i)
            last_end_time = end_times[i]

    return selected
```

Activity	Start	End
0	0	6
1	5	9
2	8	9
3	1	2
4	3	4
5	5	7

activities = [3, 4, 0, 5, 1, 2]

selected = [3, 4]

last_end_time = 4

Content Copyright Nanyang Technological University

Q2 (Normal) – Answer

```
def activity_selection(self, start_times: List[int],
                       end_times: List[int]) -> List[int]:
    activities = list(range(len(start_times)))
    activities.sort(key=lambda i: end_times[i])

    selected = []
    last_end_time = -1

    for i in activities:
        if start_times[i] >= last_end_time:
            selected.append(i)
            last_end_time = end_times[i]

    return selected
```

Activity	Start	End
0	0	6
1	5	9
2	8	9
3	1	2
4	3	4
5	5	7

activities = [3, 4, 0, 5, 1, 2]

selected = [3, 4]

last_end_time = 4

Content Copyright Nanyang Technological University

Q2 (Normal) – Answer

```
def activity_selection(self, start_times: List[int],
                       end_times: List[int]) -> List[int]:
    activities = list(range(len(start_times)))
    activities.sort(key=lambda i: end_times[i])

    selected = []
    last_end_time = -1

    for i in activities:
        if start_times[i] >= last_end_time:
            selected.append(i)
            last_end_time = end_times[i]

    return selected
```

Activity	Start	End
0	0	6
1	5	9
2	8	9
3	1	2
4	3	4
5	5	7

activities = [3, 4, 0, 5, 1, 2]

selected = [3, 4, 5]

last_end_time = 7

Content Copyright Nanyang Technological University

Q2 (Normal) – Answer

```
def activity_selection(self, start_times: List[int],
                       end_times: List[int]) -> List[int]:
    activities = list(range(len(start_times)))
    activities.sort(key=lambda i: end_times[i])

    selected = []
    last_end_time = -1

    for i in activities:
        if start_times[i] >= last_end_time:
            selected.append(i)
            last_end_time = end_times[i]

    return selected
```

Activity	Start	End
0	0	6
1	5	9
2	8	9
3	1	2
4	3	4
5	5	7

activities = [3, 4, 0, 5, 1, 2]

selected = [3, 4, 5]

last_end_time = 7

Content Copyright Nanyang Technological University

Q2 (Normal) – Answer

```
def activity_selection(self, start_times: List[int],
                       end_times: List[int]) -> List[int]:
    activities = list(range(len(start_times)))
    activities.sort(key=lambda i: end_times[i])

    selected = []
    last_end_time = -1

    for i in activities:
        if start_times[i] >= last_end_time:
            selected.append(i)
            last_end_time = end_times[i]

    return selected
```

Activity	Start	End
0	0	6
1	5	9
2	8	9
3	1	2
4	3	4
5	5	7

activities = [3, 4, 0, 5, 1, 2]

selected = [3, 4, 5, 2]

last_end_time = 9

Content Copyright Nanyang Technological University

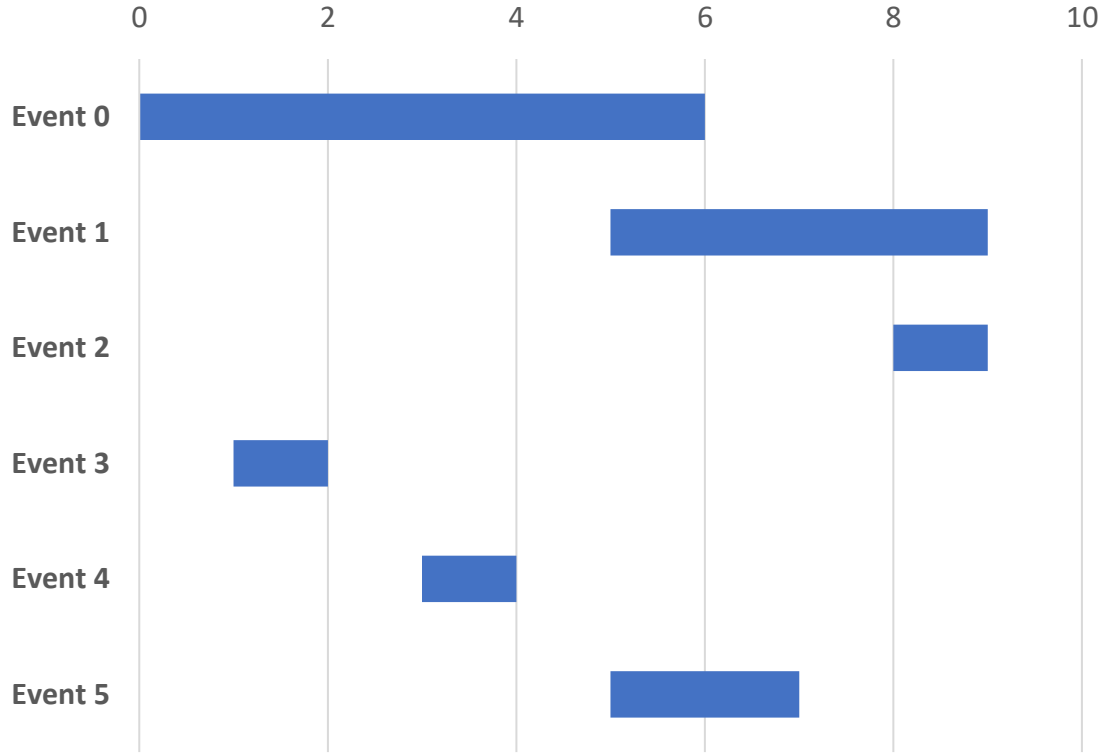
Q2 (Normal) – Answer

```
def activity_selection(self, start_times: List[int],
                       end_times: List[int]) -> List[int]:
    activities = list(range(len(start_times)))
    activities.sort(key=lambda i: end_times[i])

    selected = []
    last_end_time = -1

    for i in activities:
        if start_times[i] >= last_end_time:
            selected.append(i)
            last_end_time = end_times[i]

    return selected
```



activities = [3, 4, 0, 5, 1, 2]

selected = [3, 4, 5, 2]

last_end_time = 9

Content Copyright Nanyang Technological University

Q2 (Normal) – Answer

```
def activity_selection(self, start_times: List[int],
                       end_times: List[int]) -> List[int]:
    activities = list(range(len(start_times)))
    activities.sort(key=lambda i: end_times[i])

    selected = []
    last_end_time = -1

    for i in activities:
        if start_times[i] >= last_end_time:
            selected.append(i)
            last_end_time = end_times[i]

    return selected
```

O(n)

Content Copyright Nanyang Technological University

Q3 (Driver) – Answer

Suppose you are programming an autonomous delivery drone that must deliver packages to multiple locations in a city and return to its starting point (warehouse). The drone must find the shortest possible route that visits each delivery location exactly once before returning to the warehouse. Each location has a distance to every other location. The drone's objective is to minimize the total travel distance. Your task is to implement a brute-force approach to find the shortest distance for the inputs.

Input:

A single distance matrix of size n × n, where distance_matrix[i][j] represents the distance from location i to location j. Location 0 is always the warehouse.

For example, given 4 locations (including the warehouse):

[[0, 10, 15, 20], [10, 0, 35, 25], [15, 35, 0, 30], [20, 25, 30, 0]]

Row 0: distances from the warehouse, 0 to itself, 10 to location 1, 15 to location 2, 20 to location 3. Row 1: distances from location 1, 10 to warehouse, 0 to itself, 35 to location 2, 25 to location 3. and so on...

Output:

A list containing two elements:

The shortest total distance The best route (starting and ending at location 0)

For the example above:
[80, [0, 1, 3, 2, 0]]

The optimal route is 0 → 1 → 3 → 2 → 0 with a total distance of 10 + 25 + 30 + 15 = 80.

Complete the brute_force_tsp function and return a list containing the following values: [**min_distance**, **best_route**]

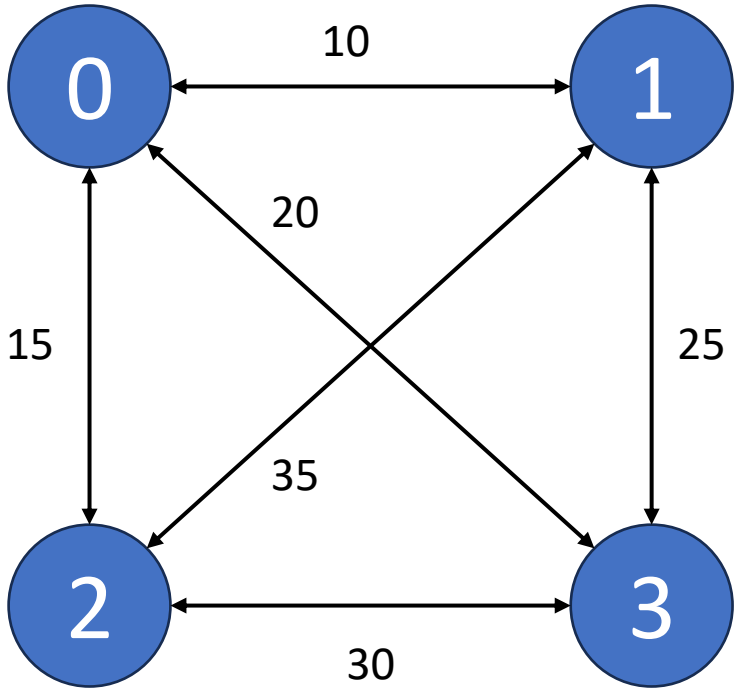
Content Copyright Nanyang Technological University

Q3 (Driver) – Answer

- Brute force approach
  - Permutations
  - P(n, r) = n!/(n-r)!
  - n! different scenarios

Route	Total Distance
0 → 1 → 2 → 3 → 0	
0 → 1 → 3 → 2 → 0	
0 → 2 → 1 → 3 → 0	
0 → 2 → 3 → 1 → 0	
0 → 3 → 1 → 2 → 0	
0 → 3 → 2 → 1 → 0	





Content Copyright Nanyang Technological University

Q3 (Driver) – Answer

```
def brute_force_tsp(self, distance_matrix):
    n = len(distance_matrix)
    locations = list(range(n))
    min_distance = float('inf')
    best_route = None

    for perm in itertools.permutations(locations[1:]):
        route = [0] + list(perm) + [0]
        total_distance = 0
        for i in range(len(route) - 1):
            total_distance += distance_matrix[route[i]][route[i + 1]]
        if total_distance < min_distance:
            min_distance = total_distance
            best_route = route

    return [min_distance, best_route]
```

Content Copyright Nanyang Technological University

Q3 (Driver) – Answer

```
def brute_force_tsp(self, distance_matrix):
    n = len(distance_matrix)
    locations = list(range(n))
    min_distance = float('inf')
    best_route = None

    for perm in itertools.permutations(locations[1:]):
        route = [0] + list(perm) + [0]
        total_distance = 0
        for i in range(len(route) - 1):
            total_distance += distance_matrix[route[i]][route[i + 1]]
        if total_distance < min_distance:
            min_distance = total_distance
            best_route = route

    return [min_distance, best_route]
```

itertools.permutations([1, 2, 3])

Output:
[(1, 2, 3), (1, 3, 2), (2, 1, 3), (2, 3, 1), (3, 1, 2), (3, 2, 1)]

Content Copyright Nanyang Technological University

Q3 (Driver) – Answer

```
def brute_force_tsp(self, distance_matrix):
    n = len(distance_matrix)
    locations = list(range(n))
    min_distance = float('inf')
    best_route = None

    for perm in itertools.permutations(locations[1:]):
        route = [0] + list(perm) + [0]
        total_distance = 0
        for i in range(len(route) - 1):
            total_distance += distance_matrix[route[i]][route[i + 1]]
        if total_distance < min_distance:
            min_distance = total_distance
            best_route = route

    return [min_distance, best_route]
```

itertools.permutations([1, 2, 3])

Output:
[(1, 2, 3), (1, 3, 2), (2, 1, 3), (2, 3, 1), (3, 1, 2), (3, 2, 1)]

Route
0 → 1 → 2 → 3 → 0
0 → 1 → 3 → 2 → 0
0 → 2 → 1 → 3 → 0
0 → 2 → 3 → 1 → 0
0 → 3 → 1 → 2 → 0
0 → 3 → 2 → 1 → 0

Content Copyright Nanyang Technological University

Q3 (Driver) – Answer

```
def brute_force_tsp(self, distance_matrix):
    n = len(distance_matrix)
    locations = list(range(n))
    min_distance = float('inf')
    best_route = None

    for perm in itertools.permutations(locations[1:]):
        route = [0] + list(perm) + [0]
        total_distance = 0
        for i in range(len(route) - 1):
            total_distance += distance_matrix[route[i]][route[i + 1]]
        if total_distance < min_distance:
            min_distance = total_distance
            best_route = route

    return [min_distance, best_route]
```

Initialise variables

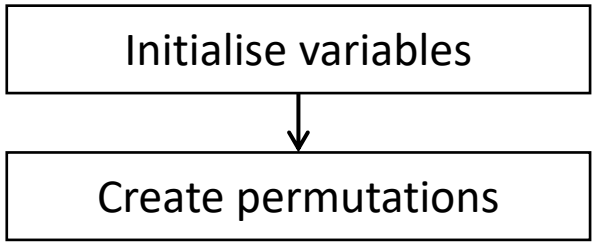
Content Copyright Nanyang Technological University

Q3 (Driver) – Answer

```
def brute_force_tsp(self, distance_matrix):
    n = len(distance_matrix)
    locations = list(range(n))
    min_distance = float('inf')
    best_route = None

    for perm in itertools.permutations(locations[1:]):
        route = [0] + list(perm) + [0]
        total_distance = 0
        for i in range(len(route) - 1):
            total_distance += distance_matrix[route[i]][route[i + 1]]
        if total_distance < min_distance:
            min_distance = total_distance
            best_route = route

    return [min_distance, best_route]
```



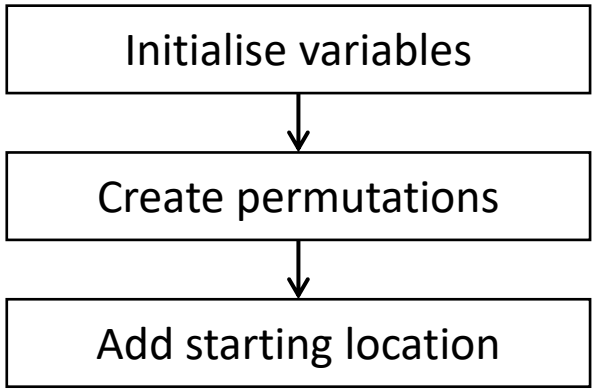
Content Copyright Nanyang Technological University

Q3 (Driver) – Answer

```
def brute_force_tsp(self, distance_matrix):
    n = len(distance_matrix)
    locations = list(range(n))
    min_distance = float('inf')
    best_route = None

    for perm in itertools.permutations(locations[1:]):
        route = [0] + list(perm) + [0]
        total_distance = 0
        for i in range(len(route) - 1):
            total_distance += distance_matrix[route[i]][route[i + 1]]
        if total_distance < min_distance:
            min_distance = total_distance
            best_route = route

    return [min_distance, best_route]
```



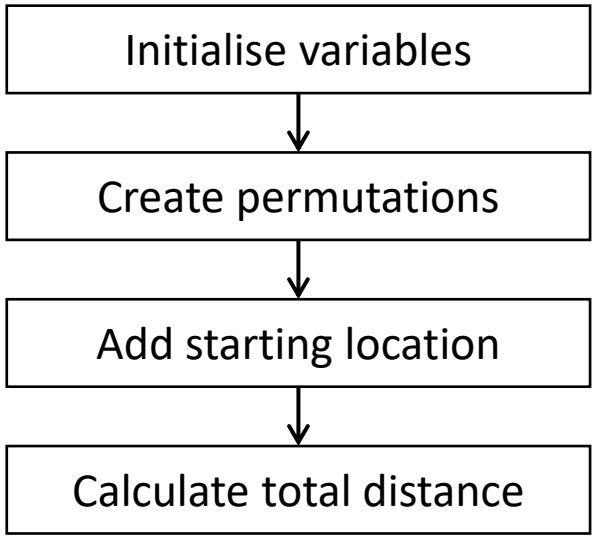
Content Copyright Nanyang Technological University

Q3 (Driver) – Answer

```
def brute_force_tsp(self, distance_matrix):
    n = len(distance_matrix)
    locations = list(range(n))
    min_distance = float('inf')
    best_route = None

    for perm in itertools.permutations(locations[1:]):
        route = [0] + list(perm) + [0]
        total_distance = 0
        for i in range(len(route) - 1):
            total_distance += distance_matrix[route[i]][route[i + 1]]
        if total_distance < min_distance:
            min_distance = total_distance
            best_route = route

    return [min_distance, best_route]
```



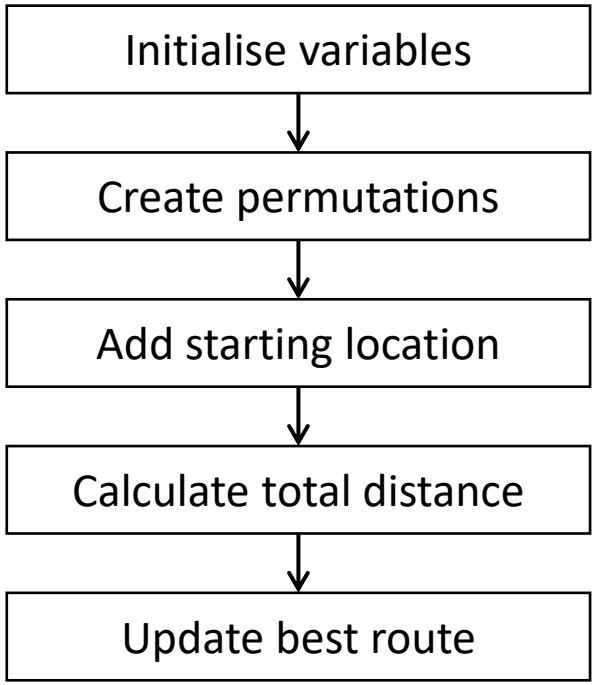
Content Copyright Nanyang Technological University

Q3 (Driver) – Answer

```
def brute_force_tsp(self, distance_matrix):
    n = len(distance_matrix)
    locations = list(range(n))
    min_distance = float('inf')
    best_route = None

    for perm in itertools.permutations(locations[1:]):
        route = [0] + list(perm) + [0]
        total_distance = 0
        for i in range(len(route) - 1):
            total_distance += distance_matrix[route[i]][route[i + 1]]
        if total_distance < min_distance:
            min_distance = total_distance
            best_route = route

    return [min_distance, best_route]
```



Content Copyright Nanyang Technological University

Q4 (Navigator) – Answer

Suppose you are a treasure hunter exploring an ancient cave filled with valuable artifacts. You have a backpack with a limited weight capacity W, and you must decide which artifacts to take to maximize your total value. Each artifact has a weight and a value. You cannot take a fraction of an artifact, you must either take it entirely or leave it behind.

Your task is to implement a brute-force approach that tries all possible subsets of artifacts to find the combination that maximizes total value while staying within the weight limit.

Input

Three parameters:

- W (int) - the backpack's weight capacity values
- (list of int) - the value of each artifact weights
- (list of int) - the weight of each artifact

For example, given W = 50, values = [60, 100, 120, 30, 80], weights = [10, 20, 30, 5, 15]:

Artifact	Value	Weight
0	60	10
1	100	20
2	120	30
3	30	5
4	80	15

Output A list containing three elements:

- The maximum total value achievable
- A list of [weight, value] pairs for each selected artifact
- The total weight used

For the example above:

[270, [[10, 60], [20, 100], [5, 30], [15, 80]], 50] The optimal selection is artifacts 0, 1, 3, and 4 with a total value of 60 + 100 + 30 + 80 = 270 and a total weight of 10 + 20 + 5 + 15 = 50.

Content Copyright Nanyang Technological University

Q4 (Navigator) – Answer

- Brute force approach
  - Each item has two states (take/don't take)
  - n^2 different scenarios

Take	Don't Take	Total Weight	Total Value
	A B C		
A	B C		
B	A C		
C	A B		
A B	C		
A C	B		
B C	A		
A B C			



Content Copyright Nanyang Technological University

Q4 (Navigator) – Answer

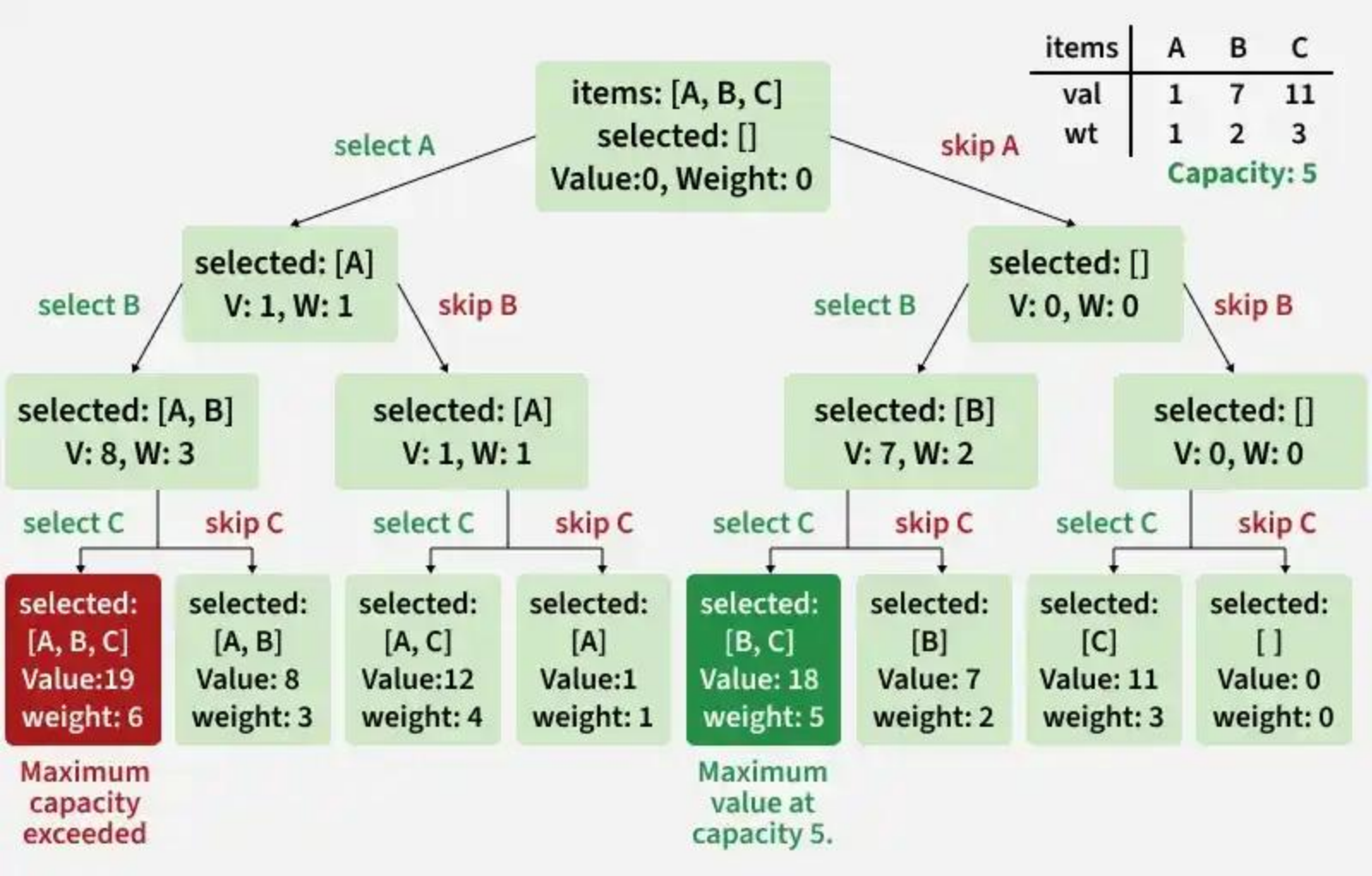
```
def brute_force_knapsack(self, weights, values, W):
    n = len(weights)
    best = [0, []]  # [best value so far, indices of the items that give it]

    def try_items(i, chosen, total_weight, total_value):
        # Every item is either left out or taken: two branches per item.
        if i == n:
            if total_value > best[0]:
                best[0], best[1] = total_value, chosen[:]
            return
        # Option 1: leave item i out
        try_items(i + 1, chosen, total_weight, total_value)
        # Option 2: take item i, if it still fits
        if total_weight + weights[i] <= W:
            chosen.append(i)
            try_items(i + 1, chosen, total_weight + weights[i], total_value + values[i])
            chosen.pop()

    try_items(0, [], 0, 0)
    selected_items = [[weights[i], values[i]] for i in best[1]]
    total_selected_weight = sum(weights[i] for i in best[1])
    return [best[0], selected_items, total_selected_weight]
```

Content Copyright Nanyang Technological University

Q4 (Navigator) – Answer



Content Copyright Nanyang Technological University

Q4 (Navigator) – Answer

```
def brute_force_knapsack(weights, values, W):
    n = len(weights)
    max_value = 0
    best_combination = []

    for combination in itertools.product([0, 1], repeat=n):
        total_weight = sum(weights[i] for i in range(n) if combination[i] == 1)
        total_value = sum(values[i] for i in range(n) if combination[i] == 1)

        if total_weight <= W and total_value > max_value:
            max_value = total_value
            best_combination = combination

    selected_items = [(weights[i], values[i]) for i in range(n) if best_combination[i] == 1]
    total_selected_weight = sum(item[0] for item in selected_items)

    return max_value, selected_items, total_selected_weight
```

Content Copyright Nanyang Technological University

Q4 (Navigator) – Answer

```
def brute_force_knapsack(weights, values, W):
    n = len(weights)
    max_value = 0
    best_combination = []

    for combination in itertools.product([0, 1], repeat=n):
        total_weight = sum(weights[i] for i in range(n) if combination[i] == 1)
        total_value = sum(values[i] for i in range(n) if combination[i] == 1)

        if total_weight <= W and total_value > max_value:
            max_value = total_value
            best_combination = combination

    selected_items = [(weights[i], values[i]) for i in range(n) if best_combination[i] == 1]
    total_selected_weight = sum(item[0] for item in selected_items)

    return max_value, selected_items, total_selected_weight
```

itertools.product([0, 1], repeat = 3)

Output:
[(0, 0, 0), (0, 0, 1), (0, 1, 0), (1, 0, 0), (1, 0, 1), (1, 1, 0), (0, 1, 1), (1, 1, 1)]

Content Copyright Nanyang Technological University

Q4 (Navigator) – Answer

Initialise 3 variables, n as the number of artifacts, best_value as 0, and best_items as an empty list

Create a recursive function with the arguments i, chosen, total_weight, total_value

Within the recursive function, the base case is i == n, in which case check if the total_value is greater than the best_value, and if so, update both the best_value and best_items based on total_value and chosen. return nothing at the end of the base case

After the base case, there are 2 recursion cases, one where item i is not taken, so the recursion is called by just incrementing i by 1. The other recursion case is where item i gets taken if the total_weight does not exceed the weight limit W. If it does not exceed, append i to chosen, recurse with i incremented by 1 and update the new total_weight and total_value, before popping chosen after the recursion

After the recursion function, call it with the initial arguments 0, [], 0, 0

Then create a list of selected_items as [weight, value] pairs based on best_items and calculate the total_selected_weight of the selected items.

Finally, return best_value, selected_items, total_selected_weight

Content Copyright Nanyang Technological University

Q4 (Navigator) – Answer

```
def brute_force_knapsack(self, weights, values, W):
    n = len(weights)
    best_value = 0
    best_items = []

    def helper(i, chosen, total_weight, total_value):
        nonlocal best_value, best_items
        if i == n:
            if total_value > best_value:
                best_value = total_value
                best_items = chosen.copy()
            return
        helper(i + 1, chosen, total_weight, total_value)
        if total_weight + weights[i] <= W:
            chosen.append(i)
            helper(i + 1, chosen, total_weight + weights[i], total_value + values[i])
            chosen.pop()

    helper(0, [], 0, 0)
    selected_items = []
    total_selected_weight = 0
    for i in best_items:
        selected_items.append([weights[i], values[i]])
        total_selected_weight += weights[i]
    return best_value, selected_items, total_selected_weight
```

Content Copyright Nanyang Technological University